

Amrit Chhaya Foundation

Backend API Documentation

API Version: v1 **Base URL:** <https://api.amritchhayafoundation.org/api/v1>

This document defines the API contract between the Amrit Chhaya Foundation frontend and backend.

1. Technology Stack

Backend

- Node.js
- Express.js
- MySQL
- REST API
- JWT Authentication

Integrations

- Razorpay — Payments
- Email Service — Email notifications
- Cloud Storage — Images/Documents

2. API Conventions

All APIs use:

Content-Type: application/json

Successful responses follow:

```
{
  "success": true,
  "message": "Operation successful",
  "data": {}
}
```

Failed responses follow:

```
{
  "success": false,
  "message": "Something went wrong",
  "error": {
    "code": "ERROR_CODE",
    "details": {}
  }
}
```

3. HTTP Status Codes

Status	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Resource Not Found
409	Conflict
422	Validation Error
429	Too Many Requests
500	Internal Server Error

4. Authentication

Admin APIs require authentication.

Authentication header:

```
Authorization: Bearer <ACCESS_TOKEN>
```

Example:

```
Authorization: Bearer eyJhbGciOi...
```

Public APIs do not require authentication unless explicitly mentioned.

5. Authentication APIs

5.1 Admin Login

```
POST /auth/login
```

Request

```
{
  "email": "admin@example.com",
  "password": "password"
}
```

Response

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "accessToken": "ACCESS_TOKEN",
    "refreshToken": "REFRESH_TOKEN",
    "user": {
      "id": 1,
      "name": "Admin",
      "email": "admin@example.com",
      "role": "SUPER_ADMIN"
    }
  }
}
```

5.2 Refresh Token

POST /auth/refresh

Request

```
{
  "refreshToken": "REFRESH_TOKEN"
}
```

Response

```
{
  "success": true,
  "data": {
    "accessToken": "NEW_ACCESS_TOKEN"
  }
}
```

5.3 Logout

POST /auth/logout

Authentication: Required

Request

```
{
  "refreshToken": "REFRESH_TOKEN"
}
```

Response

```
{  
  "success": true,  
  "message": "Logged out successfully"  
}
```

6. Public Website APIs

These APIs are used by the public website.

7. Pooja APIs

7.1 Get All Poojas

```
GET /poojas
```

Query Parameters

Parameter	Type	Required
page	number	No
limit	number	No
category	string	No
search	string	No
featured	boolean	No

Example:

```
GET /poojas?page=1&limit=10&featured=true
```

Response

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": 1,
        "name": "Kaal Sarp Dosh Puja",
        "slug": "kaal-sarp-dosh-puja",
        "shortDescription": "Special puja for Kaal Sarp Dosh",
        "image": "https://cdn.example.com/pooja.jpg",
        "startingPrice": 5100,
        "duration": "2 Hours",
        "featured": true
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 10,
      "total": 25,
      "totalPages": 3
    }
  }
}
```

8. Get Pooja Details

GET /poojas/:slug

Example:

GET /poojas/kaal-sarp-dosh-puja

Response

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Kaal Sarp Dosh Puja",
    "slug": "kaal-sarp-dosh-puja",
    "description": "Complete description...",
    "image": "https://cdn.example.com/pooja.jpg",
    "duration": "2 Hours",
    "category": {
      "id": 1,
      "name": "Dosha Puja"
    },
    "variants": [
      {
        "id": 1,
        "name": "Basic",
        "price": 5100
      },
      {
        "id": 2,
        "name": "With Prasad",
        "price": 7100
      }
    ],
    "features": ["Pandit Ji", "Puja Samagri", "Prasad"]
  }
}
```

9. Pooja Categories

Get Categories

GET /pooja-categories

Response

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Dosha Puja",
      "slug": "dosha-puja"
    },
    {
      "id": 2,
      "name": "Shiv Puja",
      "slug": "shiv-puja"
    }
  ]
}
```

10. Pooja Booking

10.1 Check Available Slots

GET /poojas/:poojaId/slots

Query Parameters

date=2026-09-20

Example:

GET /poojas/1/slots?date=2026-09-20

Response

```
{
  "success": true,
  "data": {
    "date": "2026-09-20",
    "slots": [
      {
        "id": 1,
        "startTime": "09:00",
        "endTime": "11:00",
        "available": true
      },
      {
        "id": 2,
        "startTime": "12:00",
        "endTime": "14:00",
        "available": false
      }
    ]
  }
}
```

11. Create Booking

POST /bookings

Request

```
{
  "poojaId": 1,
  "variantId": 2,
  "slotId": 1,
  "date": "2026-09-20",

  "devotee": {
    "name": "Jatin Jain",
    "phone": "9999999999",
    "email": "jatin@example.com",
    "gotra": "Kashyap",
    "address": {
      "line1": "Address",
      "city": "Ujjain",
      "state": "Madhya Pradesh",
      "pincode": "456001",
      "country": "India"
    }
  },

  "specialRequest": "Please perform the puja in the morning.",

  "prasadRequired": true
}
```

Response

```
{
  "success": true,
  "message": "Booking created successfully",
  "data": {
    "bookingId": "ACF-2026-000123",
    "bookingStatus": "PENDING_PAYMENT",
    "amount": 7100,
    "currency": "INR",
    "paymentRequired": true
  }
}
```

12. Payment APIs

12.1 Create Razorpay Order

POST /payments/create-order

Request

```
{  
  "bookingId": "ACF-2026-000123"  
}
```

Response

```
{  
  "success": true,  
  "data": {  
    "orderId": "order_xxxxxxxx",  
    "amount": 710000,  
    "currency": "INR",  
    "keyId": "rzp_xxxxxxxx"  
  }  
}
```

Amount is sent to Razorpay in paise.

13. Verify Payment

POST /payments/verify

Request

```
{
  "bookingId": "ACF-2026-000123",
  "razorpayOrderId": "order_xxxxxxxx",
  "razorpayPaymentId": "pay_xxxxxxxx",
  "razorpaySignature": "signature"
}
```

Response

```
{
  "success": true,
  "message": "Payment verified successfully",
  "data": {
    "bookingId": "ACF-2026-000123",
    "paymentStatus": "PAID",
    "bookingStatus": "CONFIRMED"
  }
}
```

Payment verification must always happen on the backend.

14. Payment Webhook

POST /payments/webhook

Used internally by Razorpay.

Supported events:

```
payment.captured
payment.failed
order.paid
refund.created
refund.processed
```

The frontend does not call this endpoint.

15. Booking APIs

Get Booking

```
GET /bookings/:bookingId
```

Response

```
{
  "success": true,
  "data": {
    "bookingId": "ACF-2026-000123",
    "bookingStatus": "CONFIRMED",
    "paymentStatus": "PAID",
    "pooja": {
      "name": "Kaal Sarp Dosh Puja"
    },
    "date": "2026-09-20",
    "time": "09:00 - 11:00",
    "devotee": {
      "name": "Jatin Jain"
    }
  }
}
```

16. Yatra APIs

Get Yatra Packages

```
GET /yatras
```

Response

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Ujjain Darshan",
      "slug": "ujjain-darshan",
      "duration": "2 Days / 1 Night",
      "startingPrice": 4999,
      "image": "https://cdn.example.com/yatra.jpg"
    }
  ]
}
```

17. Yatra Details

```
GET /yatras/:slug
```

Response

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Ujjain Darshan",
    "description": "Complete Ujjain Darshan package...",
    "duration": "2 Days / 1 Night",
    "startingPrice": 4999,
    "itinerary": [
      {
        "day": 1,
        "title": "Ujjain Temple Darshan",
        "description": "..."
      },
      {
        "day": 2,
        "title": "Mahakal Darshan",
        "description": "..."
      }
    ]
  }
}
```

18. Yatra Booking

POST /yatra-bookings

Request

```
{
  "yatraId": 1,
  "dateId": 5,
  "numberOfPeople": 2,

  "customer": {
    "name": "Jatin Jain",
    "phone": "9999999999",
    "email": "jatin@example.com"
  }
}
```

19. Donation APIs

Get Donation Categories

GET /donation-categories

Response

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Gau Seva",
      "slug": "gau-seva"
    },
    {
      "id": 2,
      "name": "Annadan",
      "slug": "annadan"
    },
    {
      "id": 3,
      "name": "Temple Seva",
      "slug": "temple-seva"
    }
  ]
}
```

20. Create Donation

POST /donations

Request

```
{
  "categoryId": 1,
  "amount": 1100,
  "donor": {
    "name": "Jatin Jain",
    "phone": "9999999999",
    "email": "jatin@example.com"
  },
  "anonymous": false
}
```

Response

```
{
  "success": true,
  "data": {
    "donationId": "DON-2026-000123",
    "amount": 1100,
    "paymentStatus": "PENDING",
    "paymentOrderId": "order_xxxxxxxx"
  }
}
```

21. Donation Receipt

```
GET /donations/:donationId/receipt
```

Response:

```
{
  "success": true,
  "data": {
    "donationId": "DON-2026-000123",
    "receiptUrl": "https://cdn.example.com/receipts/DON-2026-000123.pdf"
  }
}
```

22. Goshala APIs

Get Cows

```
GET /goshala/cows
```

Response

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Gauri",
      "breed": "Gir",
      "age": 6,
      "image": "https://cdn.example.com/cow.jpg",
      "adoptionAvailable": true
    }
  ]
}
```

23. Contact / Enquiry

Create Enquiry

POST /enquiries

Request

```
{
  "name": "Jatin Jain",
  "phone": "9999999999",
  "email": "jatin@example.com",
  "message": "I want information about Ujjain Darshan."
}
```

Response

```
{
  "success": true,
  "message": "Your enquiry has been submitted successfully"
}
```

24. Gallery

Get Gallery

```
GET /gallery
```

Query Parameters

```
category  
page  
limit
```

Response

```
{  
  "success": true,  
  "data": {  
    "items": [  
      {  
        "id": 1,  
        "title": "Mahakal Pooja",  
        "image": "https://cdn.example.com/image.jpg",  
        "category": "Pooja"  
      }  
    ]  
  }  
}
```

25. Blogs

Get Blogs

```
GET /blogs
```

Query Parameters

```
page  
limit  
category  
search
```

Get Blog

```
GET /blogs/:slug
```

26. Events

Get Events

```
GET /events
```

Event Details

```
GET /events/:slug
```

27. Testimonials

Get Testimonials

```
GET /testimonials
```

28. Admin APIs

All APIs under this section require:

Authorization: Bearer <ACCESS_TOKEN>

29. Admin Dashboard

GET /admin/dashboard

Response

```
{
  "success": true,
  "data": {
    "bookings": {
      "total": 248,
      "today": 12,
      "pending": 8,
      "completed": 210
    },
    "donations": {
      "total": 152,
      "today": 8,
      "amount": 485200
    },
    "revenue": {
      "today": 18500,
      "thisMonth": 245000,
      "total": 485200
    },
    "customers": {
      "total": 1240,
      "newThisMonth": 84
    }
  }
}
```

30. Admin Pooja Management

Create Pooja

```
POST /admin/poojas
```

Request

```
{
  "name": "Kaal Sarp Dosh Puja",
  "description": "Complete description...",
  "categoryId": 1,
  "duration": "2 Hours",
  "featured": true,
  "status": "ACTIVE"
}
```

Update Pooja

```
PUT /admin/poojas/:id
```

Delete Pooja

```
DELETE /admin/poojas/:id
```

31. Admin Booking Management

Get Bookings

```
GET /admin/bookings
```

Query Parameters

```
page  
limit  
search  
status  
paymentStatus  
fromDate  
toDate  
poojaId
```

Example:

```
GET /admin/bookings?status=CONFIRMED&fromDate=2026-09-01&toDate=2026-09-30
```

32. Update Booking Status

```
PATCH /admin/bookings/:id/status
```

Request

```
{  
  "status": "COMPLETED"  
}
```

Allowed statuses:

```
PENDING_PAYMENT  
CONFIRMED  
ASSIGNED  
IN_PROGRESS  
COMPLETED  
CANCELLED  
REFUNDED
```

33. Admin Customer Management

Get Customers

```
GET /admin/customers
```

Query Parameters

```
page  
limit  
search
```

Customer Details

```
GET /admin/customers/:id
```

Response should include:

```
Customer Profile  
Booking History  
Donation History  
Yatra History  
Payment History  
Notes
```

34. Admin Donation Management

Get Donations

```
GET /admin/donations
```

Filters

```
page  
limit  
search  
category  
paymentStatus  
fromDate  
toDate
```

35. Admin Yatra Management

Create Yatra

```
POST /admin/yatras
```

Update Yatra

```
PUT /admin/yatras/:id
```

Delete Yatra

```
DELETE /admin/yatras/:id
```

36. Admin Gallery Management

Upload Image

```
POST /admin/gallery
```

Content-Type:

```
multipart/form-data
```

Fields:

```
image  
title  
categoryId  
description
```

37. Admin Blog Management

Create Blog

```
POST /admin/blogs
```

Request

```
{
  "title": "Importance of Mahakal Darshan",
  "slug": "importance-of-mahakal-darshan",
  "content": "Blog content...",
  "status": "PUBLISHED",
  "seoTitle": "Importance of Mahakal Darshan",
  "seoDescription": "..."}
}
```

38. Admin Enquiries

Get Enquiries

```
GET /admin/enquiries
```

Update Enquiry

```
PATCH /admin/enquiries/:id
```

Request

```
{
  "status": "CONTACTED",
  "notes": "Customer contacted successfully."
}
```

Allowed statuses:

```
NEW
CONTACTED
FOLLOW_UP
CONVERTED
CLOSED
```

39. Admin Media Upload

```
POST /admin/media/upload
```

Content-Type:

```
multipart/form-data
```

Response

```
{
  "success": true,
  "data": {
    "url": "https://cdn.example.com/uploads/image.jpg",
    "fileName": "image.jpg"
  }
}
```

40. Notifications

The backend can send notifications for:

```
BOOKING_CREATED
PAYMENT_SUCCESS
BOOKING_CONFIRMED
BOOKING_REMINDER
POOJA_COMPLETED
PRASAD_SHIPPED
DONATION_SUCCESS
YATRA_BOOKING_CONFIRMED
NEW_ENQUIRY
```

Channels:

```
WHATSAPP
EMAIL
```

41. Booking Notification Flow



42. Error Codes

Code	Meaning
AUTH_REQUIRED	Authentication required
INVALID_TOKEN	Invalid/expired token
INVALID_CREDENTIALS	Invalid login
VALIDATION_ERROR	Invalid input
RESOURCE_NOT_FOUND	Resource does not exist
BOOKING_NOT_FOUND	Booking does not exist
SLOT_UNAVAILABLE	Selected slot unavailable
PAYMENT_FAILED	Payment failed
PAYMENT_VERIFICATION_FAILED	Payment verification failed
DUPLICATE_BOOKING	Duplicate booking
PERMISSION_DENIED	Insufficient permissions
FILE_UPLOAD_FAILED	File upload failed
SERVER_ERROR	Internal server error

43. Pagination Standard

All list APIs should use:

```
?page=1&limit=20
```

Response:

```
{
  "items": [],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 100,
    "totalPages": 5,
    "hasNext": true,
    "hasPrevious": false
  }
}
```

44. Date & Time Standard

All dates sent to the API:

```
YYYY-MM-DD
```

Example:

```
2026-09-20
```

Time:

```
HH:mm
```

Example:

```
09:30
```

Datetime:

ISO 8601

Example:

2026-09-20T09:30:00+05:30

Timezone:

Asia/Kolkata

45. Currency

All monetary values returned by the normal API are in:

INR

Example:

```
{
  "amount": 5100,
  "currency": "INR"
}
```

For Razorpay order creation, amount is converted to paise.

₹5,100 = 510000 paise

46. Frontend Integration Rules

The frontend developer must:

DO

- Use API responses instead of hardcoded data.

- Handle loading states.
- Handle API errors.
- Handle empty states.
- Store authentication token securely.
- Use pagination.
- Validate forms before submitting.
- Prevent duplicate payment submissions.

DON'T

- Calculate final booking prices on the frontend.
- Mark payments as successful based only on frontend response.
- Expose Razorpay secret keys.
- Expose database credentials.
- Call admin APIs without authentication.
- Hardcode booking IDs.
- Hardcode available slots.

47. Booking Price Rule

The frontend should send:

```
{
  "poojaId": 1,
  "variantId": 2
}
```

The backend determines the final price.

The frontend must **not** send:

```
{
  "amount": 7100
}
```

as the source of truth.

Backend:

```
Pooja
  ↓
Variant
  ↓
Database Price
  ↓
Calculate Final Amount
  ↓
Create Booking
  ↓
Create Razorpay Order
```

This prevents price manipulation.

48. API Versioning

Current version:

```
/api/v1
```

Example:

```
/api/v1/poojas
/api/v1/bookings
/api/v1/donations
```

Breaking changes should result in a new version:

```
/api/v2
```

Existing frontend integrations should continue working with v1.

49. Environment Variables

Backend environment variables:

```
PORT=5000
```

```
NODE_ENV=production
```

```
DATABASE_HOST=
```

```
DATABASE_PORT=3306
```

```
DATABASE_NAME=
```

```
DATABASE_USER=
```

```
DATABASE_PASSWORD=
```

```
JWT_ACCESS_SECRET=
```

```
JWT_REFRESH_SECRET=
```

```
RAZORPAY_KEY_ID=
```

```
RAZORPAY_KEY_SECRET=
```

```
RAZORPAY_WEBHOOK_SECRET=
```

```
WHATSAPP_ACCESS_TOKEN=
```

```
WHATSAPP_PHONE_NUMBER_ID=
```

```
WHATSAPP_VERIFY_TOKEN=
```

```
SMTP_HOST=
```

```
SMTP_PORT=
```

```
SMTP_USER=
```

```
SMTP_PASSWORD=
```

```
CLOUD_STORAGE_URL=
```

```
CLOUD_STORAGE_KEY=
```

```
CLOUD_STORAGE_SECRET=
```

Never commit .env files to GitHub.

50. Complete API Module Overview

/api/v1

```
|— /auth
|   |— login
|   |— refresh
|   └─ logout
|
|— /poojas
|   |— GET /
|   |— GET /:slug
|   └─ GET /:id/slots
|
|— /bookings
|   |— POST /
|   └─ GET /:bookingId
|
|— /payments
|   |— POST /create-order
|   |— POST /verify
|   └─ POST /webhook
|
|— /yatras
|   |— GET /
|   └─ GET /:slug
|
|— /yatra-bookings
|   └─ POST /
|
|— /donations
|   |— POST /
|   └─ GET /:donationId/receipt
|
|— /donation-categories
|   └─ GET /
|
|— /goshala
|   └─ GET /cows
|
|— /gallery
|   └─ GET /
|
|— /blogs
|   |— GET /
```

```
|   └─ GET /:slug
|
|─ /events
|   └─ GET /
|   └─ GET /:slug
|
|─ /testimonials
|   └─ GET /
|
|─ /enquiries
|   └─ POST /
|
└─ /admin
    └─ /dashboard
    └─ /poojas
    └─ /bookings
    └─ /customers
    └─ /donations
    └─ /yatras
    └─ /gallery
    └─ /blogs
    └─ /events
    └─ /enquiries
    └─ /media
```

51. Frontend Developer Quick Start

The frontend developer should first configure:

```
VITE_API_BASE_URL=https://api.amritchhayafoundation.org/api/v1
```

Then:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;
```

Example:

```
const response = await fetch(`${API_BASE_URL}/poojas`);

const result = await response.json();

if (result.success) {
  console.log(result.data);
}
```

For authenticated requests:

```
fetch(`${API_BASE_URL}/admin/bookings`, {
  headers: {
    Authorization: `Bearer ${accessToken}`,
    "Content-Type": "application/json",
  },
});
```

52. API Contract Rules

Both frontend and backend developers must follow this document.

If an API needs to change:

1. Discuss the change.
2. Update this documentation.
3. Update backend.
4. Inform frontend developer.
5. Test the API.
6. Update frontend integration.

Do not silently change:

- Endpoint names
 - Request fields
 - Response fields
 - Status values
 - Authentication requirements
-

53. Recommended Development Order

Backend development should follow:

1. Database Schema
- ↓
2. Authentication
- ↓
3. Poojas
- ↓
4. Slots
- ↓
5. Bookings
- ↓
6. Razorpay
- ↓
7. Customers / CRM
- ↓
8. Donations
- ↓
9. Yatras
- ↓
10. Goshala
- ↓
11. Gallery / Blog / Events
- ↓
12. WhatsApp / Email
- ↓
13. Admin Dashboard
- ↓
14. Reports
- ↓
15. Security / Audit Logs

54. API Documentation Status

Module	Status
--------	--------

Module	Status
Authentication	Planned
Poojas	Planned
Pooja Slots	Planned
Bookings	Planned
Payments	Planned
Customers	Planned
Donations	Planned
Yatras	Planned
Goshala	Planned
Gallery	Planned
Blogs	Planned
Events	Planned
Testimonials	Planned
Enquiries	Planned
WhatsApp	Planned
Email	Planned
Admin	Planned
Reports	Planned

Document Version: 1.0.0 **Last Updated:** 2026-09-11

